

书脉项目架构拆解文档

1. 文档目标

这份文档围绕两个目的来写：

- 1. 你可以根据这份架构说明，按模块把这个项目重新复刻出来。
- 2. 你可以在答辩时说清楚每个模块“干什么、为什么要有、没有它会怎样”。

项目在代码层的技术名是 `smart-library`，在产品表达层的名字是：

书脉，基于传承笔记的图书知识链路平台

它不是一个单纯的“图书借还管理系统”，而是把下面三类能力合并到了一个平台里：

- 1. 传统图书馆业务能力：登录、读者管理、图书管理、借阅归还、续借审批、统计分析。
- 2. 知识沉淀能力：读书笔记、公开笔记广场、传承笔记。
- 3. 智能服务能力：AI 对话、语义检索、工具调用式借书推荐与馆藏问答。

2. 一句话概括整个系统

这个项目采用 **Vue3 + Express + SQLite + OpenAI 兼容 AI 接口** 的前后端分离架构，前端负责界面交互和流程编排，后端负责业务规则、权限校验和数据持久化，数据库负责保存图书馆核心业务数据，AI 模块通过向量检索和工具调用把“问答”升级成“可执行图书馆助手”。

3. 技术栈总表

层级	技术	作用
前端框架	Vue 3 + TypeScript	构建单页应用，负责页面、状态、交互
UI 组件	Element Plus	表单、表格、对话框、消息提示等基础组件
状态管理	Pinia	保存当前用户、Token、登录状态
路由	Vue Router	登录页、首页、图书页、借阅页、AI 页等页面切换
图表	ECharts	统计分析页面展示借阅趋势、分类统计等
HTTP 通信	Axios	前端统一请求后端 API
后端框架	Express + TypeScript	提供 REST API、鉴权、业务处理
数据库	SQLite + better-sqlite3	保存用户、读者、图书、借阅、笔记、通知、AI 配置等
密码加密	bcryptjs	登录密码校验
身份认证	JWT	登录态验证、Token 刷新
安全防护	helmet、cors、限流中间件	请求安全、跨域控制、防爆破

层级	技术	作用
AI SDK	openai	调用兼容 OpenAI 的聊天和 embedding 接口
构建工具	Vite	打包前端
运行工具	tsx	直接运行 TypeScript 后端

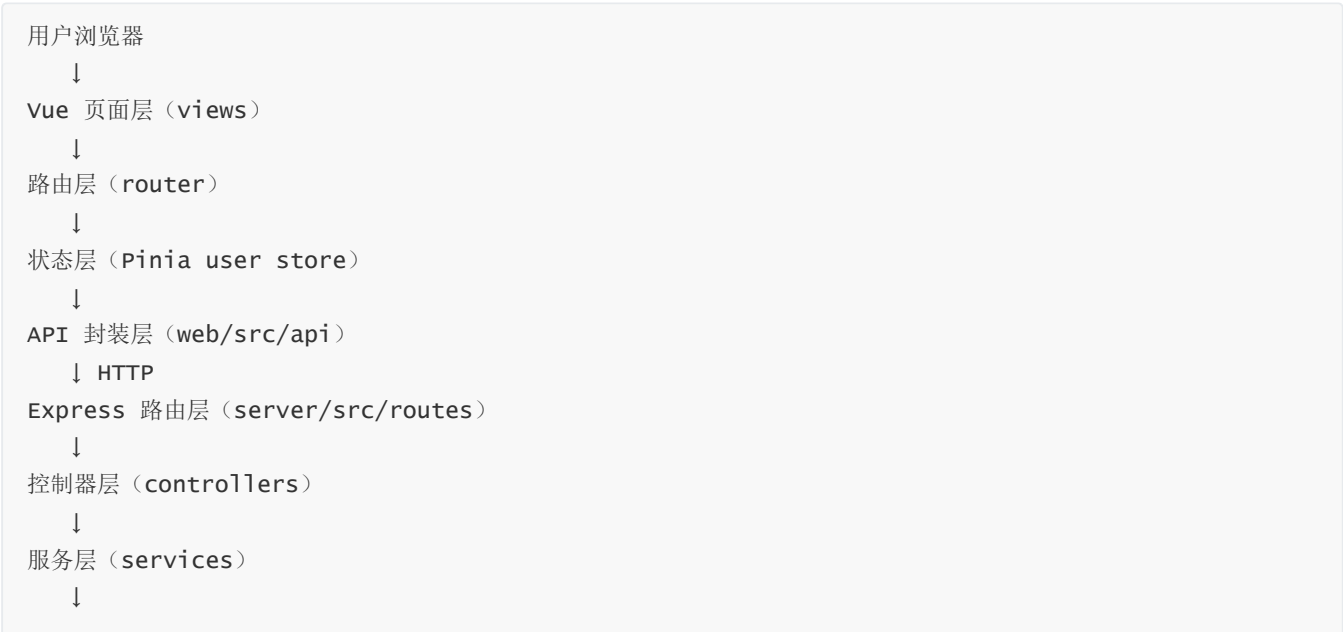
4. 根目录结构与职责

```
smart-library/
├─ data/           # SQLite 数据库文件等运行数据
├─ scripts/        # 数据生成、清库、启动脚本
├─ server/         # 后端 Express + TypeScript
├─ web/            # 前端 Vue3 + TypeScript
├─ .env / .env.example # 环境变量示例与本地配置
├─ package.json    # 根脚本，统一管理前后端运行入口
├─ start.bat       # windows 快速启动
├─ start.sh        # Linux/macOS 快速启动
└─ README.md       # 项目说明
```

4.1 每个根目录为什么存在

- `web/`：承载全部用户界面。没有它，系统就只有 API，没有可以展示给评委看的可视化产品。
- `server/`：承载全部业务规则。没有它，前端只能做假页面，无法真正完成借阅、续借、权限控制。
- `data/`：让系统数据真正落地。没有它，所有图书、读者、借阅记录都无法持久保存。
- `scripts/`：解决“项目如何初始化和演示”的问题。没有它，第一次运行、演示准备、生成样例数据会很麻烦。
- `.env`：管理端口、AI Key、跨域来源等环境配置。没有它，部署时会把配置硬编码进代码，不利于迁移。

5. 系统总架构图



仓储层 (repositories)



SQLite 数据库

AI 链路额外分支:

前端 AI 页面



后端 AI 路由



runAgentLoop



工具调用 executeTool



数据库 / BorrowingService / NoteRepository / 向量检索 / 外部模型 API

5.1 这个分层为什么合理

- 前端只负责“展示和交互”，不负责最终业务裁决。
- 后端只负责“规则和数据”，不会和页面样式耦合。
- 数据库只负责“持久化事实”，例如谁借了哪本书、什么时候到期。
- AI 被单独做成一条链路，而不是直接写在某个页面里，这样后面你可以独立升级模型、工具、提示词。

答辩时可以这样说：

我们采用的是典型的前后端分层架构。前端负责交互体验，后端负责业务规则和权限控制，数据库负责持久化，AI 模块在后端独立成层，通过工具调用接入真实馆藏数据，这样可以保证系统既好用，又能保证数据一致性和安全性。

6. 前端架构拆解

前端不是“很多页面拼起来”这么简单，而是按照 **启动层 -> 路由层 -> 状态层 -> API 层 -> 页面层 -> 组件层** 来组织的。

6.1 启动层：web/src/main.ts

这里是前端入口。

它负责做几件事：

1. 创建 Vue 应用。
2. 注册 Pinia。
3. 注册 Vue Router。
4. 注册 Element Plus。
5. 挂载根组件 App.vue。

6.2 根组件层: `web/src/App.vue`

它的作用不是写具体业务，而是提供全局承载容器，例如：

- 路由视图出口；
- 全局样式入口；
- 全局页面外壳。

它存在的意义是把“具体页面”和“应用根容器”分开，便于以后扩展全局主题、全局通知、全局弹窗。

6.3 路由层: `web/src/router/index.ts`

这个文件是整个前端流程的总调度中心。

它定义了这些核心路由：

- `/login`
- `/register`
- `/dashboard`
- `/books`
- `/readers`
- `/borrowing`
- `/statistics`
- `/notes`
- `/ai-assistant`
- `/settings`

它最关键的不是“声明页面路径”，而是 `router.beforeEach()` 路由守卫。

这个守卫做了三件大事：

1. 首次访问时调用 `userStore.initialize()` 恢复会话。
2. 检查页面是否需要登录。
3. 检查当前角色是否有权限进入某个页面。

也就是说，前端第一道“访问控制”是在这里做的。

6.4 状态层: `web/src/store/user.ts`

这是前端登录态中心。

它保存：

- `user`
- `token`
- `isInitialized`
- `isLoggedIn`

它暴露的核心函数有：

- `login()`
- `logout()`
- `restoreSession()`
- `validateToken()`
- `initialize()`
- `clearSession()`
- `updateToken()`

这个模块是干什么的

它负责把“用户登录状态”从一个临时页面行为，升级为全局共享状态。

为什么一定要有

如果没有这个模块：

- 每个页面都要自己判断登录态；
- 每个页面都要自己处理 token；
- 页面刷新后会直接丢失登录状态。

这个设计的答辩表达

我们把用户状态集中放在 Pinia 里管理，并且结合 localStorage 做会话恢复。这样页面刷新后不需要重新登录，同时也保证了路由守卫、请求拦截器、页面展示都能使用同一份用户信息。

6.5 API 层： `web/src/api/*`

前端所有请求都没有散落在页面里，而是统一封装到 API 层。

核心文件包括：

- `web/src/api/index.ts`
- `web/src/api/auth.api.ts`
- `web/src/api/reader.api.ts`
- `web/src/api/book.api.ts`
- `web/src/api/borrowing.api.ts`
- `web/src/api/ai.api.ts`
- `web/src/api/notes.api.ts`
- `web/src/api/notification.api.ts`
- `web/src/api/other.api.ts`

其中 `web/src/api/index.ts` 是所有请求的总入口。

它做的关键事情有：

1. 创建 Axios 实例。

2. 在请求拦截器里自动携带 `Authorization: Bearer token`。
3. 在响应拦截器里处理 `X-New-Token` 自动刷新。
4. 401 时尝试调用 `/auth/refresh` 刷新 Token。
5. 刷新失败后清空本地状态并重定向回登录页。

为什么 API 层必须抽出来

因为页面的职责应该是“用户交互”，不是“拼接 URL、处理重试、处理 token 刷新”。把 API 层抽出来之后，页面代码更清晰，也更容易维护。

6.6 页面层： `web/src/views/*`

页面层就是用户能直接看到的业务页面。每个页面都对应一个清晰业务领域。

`Login.vue`

作用：

- 输入账号密码；
- 调用 `handleLogin()` 发起登录；
- 登录成功后跳转首页或原目标页面。

存在意义：

- 把“身份进入系统”做成明确入口；
- 为后面的角色权限体系提供基础。

`Dashboard.vue`

作用：

- 展示系统概览；
- 让用户进入系统后先看到总体状态。

存在意义：

- 它不是必须业务页面，但它提升了系统专业感和导航效率。

`Books.vue`

作用：

- 图书检索、分类筛选、图书列表展示；
- 管理员/馆员可以新增、编辑、删除、导出图书；
- 教师/学生可以直接点击“借阅”。

存在意义：

- 它是系统的“馆藏入口页”；
- 同时还是普通用户发起借书的一个主入口。

Readers.vue

作用：

- 管理读者；
- 维护读者类别；
- 挂失、激活、续期、统计。

存在意义：

- 现实图书馆里借书资格不是只看登录账号，而是看“读者身份”；
- 所以必须把读者业务单独建模。

Borrowing.vue

作用：

- 馆员/管理员执行借书、还书、查看借阅记录；
- 普通读者查看自己的当前借阅、历史记录；
- 普通读者发起续借申请；
- 管理端处理借阅相关管理动作。

存在意义：

- 它是传统图书管理系统的核心业务页。

Notes.vue

作用：

- 我的笔记；
- 公开笔记广场；
- 传承笔记查看与发布。

存在意义：

- 这是项目和普通图书管理系统拉开差距的关键模块；
- 它把“借书”升级为“借书 + 读书经验传递”。

AIAssistant.vue

作用：

- 与 AI 对话；
- 查看推荐图书；
- 从推荐结果直接借阅；
- 保存和加载会话历史。

存在意义：

- 它不是装饰性的聊天框，而是一个能调用系统真实数据的智能入口。

Settings.vue

作用：

- 管理 AI 配置；
- 测试 AI 服务是否可用。

存在意义：

- 把 AI 配置从代码里解耦出来，支持后台运维。

Statistics.vue

作用：

- 借阅趋势、热门图书、活跃读者等统计可视化。

存在意义：

- 给系统增加“决策支持”能力，不只是事务处理。

6.7 组件层：web/src/components/*

Layout.vue

作用：

- 统一侧边菜单、顶部区域、页面框架。

引入目的：

- 避免每个页面重复写导航和整体布局。

NotificationCenter.vue

作用：

- 展示通知；
- 展示待审批续借请求；
- 支持管理员广播通知；
- 支持标记已读。

引入目的：

- 解决“续借申请是异步流程”这个问题；
- 把原本分散的提醒、审批、广播收拢到一个中心。

7. 后端架构拆解

后端采用的是 启动层 -> 应用装配层 -> 中间件层 -> 路由层 -> 控制器层 -> 服务层 -> 仓储层 -> 数据层 的结构。

7.1 启动层: `server/src/server.ts`

这是后端真正启动的入口。

它按顺序做了四件关键事：

1. `setupDatabase()`：初始化数据库。
2. `autoSeedIfEmpty(db)`：空库时自动灌入演示数据。
3. `getAIConfig()`：检测 AI 配置是否存在。
4. `createApp()` 后 `app.listen(PORT)`：启动 Express 服务。

为什么启动层要这样设计

因为系统不是“服务一启动就能跑”，而是依赖：

- 数据库已经建表；
- 默认权限已存在；
- AI 配置已可读取；
- 演示环境有基础数据。

7.2 应用装配层: `server/src/app.ts`

这个文件负责把整个 Express 应用拼起来。

它做的事包括：

1. 开启 `helmet` 安全头。
2. 配置自定义 CORS 允许来源。
3. 开启 JSON 解析。
4. 挂载 `globalLimiter` 和 `apiLimiter`。
5. 挂载 `auditMiddleware`。
6. 提供 `/health` 健康检查。
7. 统一挂载 `/api/v1/*` 路由。
8. 如果 `web/dist` 存在，就静态托管前端打包产物。
9. 最后挂载 404 和统一错误处理中间件。

这个文件的本质

它就是后端的“总装车间”。

真正的业务规则不在这里，但所有能力都必须从这里被装配起来。

7.3 中间件层

中间件层负责把“公共能力”从业务代码里抽出来。

核心中间件有：

- `auth.middleware.ts`

- `permission.middleware.ts`
- `rateLimit.middleware.ts`
- `validation.middleware.ts`
- `audit.middleware.ts`
- `error.middleware.ts`

`auth.middleware.ts`

作用:

- 解析 Bearer Token;
- 调用 `verifyToken()`;
- 从数据库查当前用户;
- 写入 `req.user`;
- 如果 token 快过期, 使用 `generateToken()` 生成新 token, 并通过 `X-New-Token` 返回。

引入目的:

- 统一身份认证;
- 避免每个控制器都重复解析 token。

`permission.middleware.ts`

作用:

- 通过 `requirePermission()` 和 `requireRole()` 做 RBAC 权限控制。

引入目的:

- 防止“登录了就什么都能做”;
- 把系统从身份认证升级成权限治理。

`rateLimit.middleware.ts`

作用:

- 限制登录爆破;
- 限制高频接口滥用。

`validation.middleware.ts`

作用:

- 校验登录、注册、改密等请求格式。

`audit.middleware.ts`

作用:

- 记录敏感操作审计轨迹。

`error.middleware.ts`

作用：

- 把不同业务异常统一转成前端可识别的错误响应。

7.4 路由层

路由层负责“把 URL 分发到正确控制器”，本身不写重业务。

核心路由如下：

- `auth.routes.ts`
- `reader.routes.ts`
- `book.routes.ts`
- `borrowing.routes.ts`
- `ai.routes.ts`
- `config.routes.ts`
- `export.routes.ts`
- `search.routes.ts`
- `notification.routes.ts`
- `domains/note/note.routes.ts`

其中一个重要设计是：很多路由文件先 `router.use(authMiddleware)`，再在具体接口上挂 `requirePermission()`。

这表示系统把“先登录，再分权限”分成了两层。

7.5 控制器层

控制器层典型文件有：

- `auth.controller.ts`
- `reader.controller.ts`
- `book.controller.ts`
- `borrowing.controller.ts`
- `note.controller.ts`

控制器层的职责只有三个：

1. 接收请求参数；
2. 调服务层；
3. 返回统一 JSON 响应。

它不应该直接写复杂 SQL，也不应该直接承担完整业务规则。

7.6 服务层

服务层是后端的业务核心。

比如：

- `AuthService` 负责登录、注册、权限判断、改密码；
- `BorrowingService` 负责借书、还书、续借、审批、丢失处理；
- `NoteService` 负责私有/公开/传承笔记的业务规则；
- `ai.service.ts` 负责 AI 配置、Embedding、提示词等；
- `notification.service.ts` 负责业务通知投递。

为什么服务层最重要

因为真正的规则都在这里，例如：

- 读者是否还能借书；
- 图书库存是否足够；
- 是否有逾期；
- 传承笔记是否允许创建；
- 续借是否达到上限。

答辩时可以直接说：

控制器只负责收发请求，真正的业务约束全部在 Service 层，这样做可以避免业务逻辑分散在多个接口里，后续扩展和测试都更稳定。

7.7 仓储层

仓储层就是 repository，例如：

- `user.repository.ts`
- `reader.repository.ts`
- `book.repository.ts`
- `borrowing.repository.ts`
- `note.repository.ts`

它们负责直接操作数据库。

也就是说：

- Service 负责“应该怎么做”；
- Repository 负责“具体怎么查、怎么改库”。

7.8 AI 专用业务层

AI 相关的后端核心文件有两个：

- `server/src/domains/ai/ai.service.ts`
- `server/src/domains/ai/tools.ts`

这里的设计非常关键。

它不是简单调用大模型，而是做成了：

1. 获取 AI 配置；
2. 构造系统提示词；
3. 建立流式对话；
4. 当模型需要查数据时，触发工具调用；
5. 工具调用真实访问数据库和业务服务；
6. 把工具结果再喂回模型；
7. 最后把答案流式返回前端。

这意味着这个 AI 助手不是“只会聊天”，而是“会操作系统能力”的 Agent。

8. 数据库设计拆解

数据库是这个项目的“事实中心”。
所有页面最终都要落回数据库表结构。

8.1 核心表总览

表名	作用	为什么要单独建这张表
<code>users</code>	登录账号、角色、基础身份信息	解决“谁能进入系统”的问题
<code>reader_categories</code>	读者类别规则	不同读者借阅上限、借阅时长不同
<code>readers</code>	真实读者业务身份	解决“谁能借书、借阅规则是什么”的问题
<code>book_categories</code>	图书分类	支持分类管理和统计分析
<code>books</code>	图书主数据	系统最核心的馆藏实体
<code>borrowing_records</code>	借阅流水	记录借书、还书、逾期、罚金等事实
<code>role_permissions</code>	角色权限映射	实现 RBAC
<code>system_settings</code>	系统配置	尤其是 AI 配置后台化
<code>ai_conversations</code>	AI 会话历史	保存对话，支持继续聊
<code>operation_logs</code>	操作日志	记录业务操作痕迹
<code>audit_logs</code>	审计日志	记录敏感访问和安全审计

表名	作用	为什么要单独建这张表
book_vectors	图书向量数据	为语义检索服务，不污染图书主表
notes	笔记数据	支持私有、公开、传承三种知识沉淀模式
renewal_requests	续借审批流	支持“申请-审批-通知”的异步流程
notifications	通知中心数据	统一承载系统提醒、审批结果、广播

8.2 最需要讲清楚的几张表

users 和 readers 为什么分开

这是答辩里非常容易被问到的问题。

users 是登录身份，readers 是借阅身份。

你可以这样理解：

- users 回答的是：这个人能不能登录？他是什么角色？
- readers 回答的是：这个人能借几本书？借多久？证件是否过期？

为什么不能合一？

因为系统里有管理员、馆员，他们可能有登录账号，但并不一定是普通借阅读者。
同时普通读者的借阅规则又依赖读者类别，所以必须把“账号身份”和“借阅身份”拆开。

答辩标准回答：

我们把 users 和 readers 分离，是为了把系统登录身份和图书业务身份解耦。users 负责认证与权限，readers 负责借阅规则与读者属性，这样模型更清晰，也更符合真实图书馆业务。

reader_categories

这张表定义：

- 最大可借数量；
- 最大借阅天数；
- 有效期天数。

它的价值是把“规则”配置化，而不是写死在代码里。
比如教师和学生的借阅权限可以不同，改表数据就能变。

books

这张表保存：

- 标题、作者、ISBN、出版社；
- 分类；
- 总库存、可借库存；
- 状态；
- 描述、关键词等。

为什么要有 `available_quantity` 和 `total_quantity` 两个字段？

- `total_quantity` 表示馆藏总量；
- `available_quantity` 表示当前可借量。

这样借书时只需要减可借库存，不需要反复统计全部借阅流水，效率更高。

`borrowing_records`

这是借阅系统的流水表。

它保存：

- 谁借了哪本书；
- 借出日期；
- 应还日期；
- 实际归还日期；
- 状态；
- 续借次数；
- 罚金。

它的意义是：**借阅是一条业务流水，不是图书表上的一个字段。**

`notes`

这张表是项目创新点之一。

它最重要的字段是：

- `visibility`
- `legacy_borrowing_id`

其中 `visibility` 有三种：

- `private`
- `public`
- `legacy`

这代表三种完全不同的产品语义：

1. 私有笔记：只给自己看。
2. 公开笔记：进入广场，大家都能看。
3. 传承笔记：与某本正在借阅的书绑定，后来的借阅者可以看到前人的经验。

这不是普通 CRUD，而是把“阅读经验”做成了可流转知识。

book_vectors

这张表专门存：

- `book_id`
- `embedding_model`
- `vector`
- `text`

为什么不把向量直接放进 `books` 表？

因为向量检索属于 AI 检索层，不属于图书主数据。

拆出来后：

- 可以单独重建 `embedding`；
- 可以切换模型；
- 不影响主业务表。

renewal_requests + notifications

这两张表必须一起理解。

`renewal_requests` 负责保存“申请这件事”，

`notifications` 负责保存“有人要处理 / 处理结果是什么”。

这就把续借做成了完整审批流，而不是一个同步按钮。

8.3 乐观锁与版本字段

部分表里有 `version` 字段。

它的意义是做并发控制，避免多个管理员同时修改同一条记录时产生覆盖问题。

这是一个偏工程化、偏专业的设计点，答辩里说出来会显得你有系统设计意识。

9. 权限与认证体系

9.1 认证流程

后端认证基于 JWT。

关键文件：

- `server/src/controllers/auth.controller.ts`
- `server/src/middleware/auth.middleware.ts`
- `server/src/lib/jwt.ts`

核心过程：

1. 登录成功后生成 Token。
2. 前端把 Token 放入 `localStorage`。
3. 后续请求通过 Axios 拦截器自动带上 Token。

4. 后端 `authMiddleware` 校验 Token。
5. 如果 Token 临近过期，后端通过 `x-New-Token` 自动下发新 Token。
6. 前端响应拦截器收到后自动更新本地 Token。

这个设计的优势是：

- 用户刷新页面不会立刻掉线；
- 不需要频繁手动重新登录；
- 服务端仍然保留认证控制权。

9.2 权限体系

系统角色有四类：

- `admin`
- `librarian`
- `teacher`
- `student`

权限来自 `role_permissions` 表。

默认权限大致是：

- `admin`：全部权限
- `librarian`：图书、读者、借阅、统计的大部分管理权限
- `teacher`：可读图书、可借阅
- `student`：可读图书、可借阅

9.3 这个权限模型为什么合理

- 管理员负责系统级控制；
- 馆员负责具体馆务；
- 教师/学生负责借阅和使用服务；
- 权限不是写死在前端菜单里，而是由后端最终裁决。

这意味着：

- 前端菜单只是“体验层限制”；
- 后端权限才是“安全层限制”。

10. 从进入网页到登录：完整函数调用链

这一部分是你答辩最该背熟的内容。

10.1 第一步：用户访问网页

浏览器打开网页后，前端从 `web/src/main.ts` 启动应用。

然后路由进入 `web/src/router/index.ts`。

此时会先触发：

```
router.beforeEach()
```

在第一次访问时，它会执行：

```
userStore.initialize()
```

而 `initialize()` 内部又会走：

1. `restoreSession()`：先尝试从 `localStorage` 恢复 `token` 和 `user`。
2. 如果存在 `token`，则继续执行 `validateToken()`。
3. `validateToken()` 调用 `authApi.validate()`。
4. `authApi.validate()` 通过 `request.get('/auth/validate')` 请求后端。

10.2 如果未登录会发生什么

如果 `userStore.isLoggedIn` 为假，那么路由守卫会：

```
next({ path: '/login', query: { redirect: to.fullPath } })
```

也就是说：

- 系统会把用户重定向到登录页；
- 同时记住用户原本想去哪个页面。

10.3 用户点击登录按钮之后

登录页文件是 `web/src/views/Login.vue`。

用户点击按钮后触发：

```
handleLogin()
```

这个函数内部会按顺序执行：

1. 检查用户名是否为空；
2. 检查密码是否为空；
3. 调用 `userStore.login({ username, password })`；
4. `userStore.login()` 调用 `authApi.login(credentials)`；
5. `authApi.login()` 调用 `request.post('/auth/login')`。

10.4 后端登录链路

请求到达后端后，完整链路是：

1. `auth.routes.ts`
2. `router.post('/login', loginLimiter, validate(Schemas.login), login)`
3. `loginLimiter`：防止暴力破解
4. `validate(Schemas.login)`：校验参数格式
5. `auth.controller.login`
6. `AuthService.login()`
7. `UserRepository.findByUsername()`
8. `bcrypt.compare()` 校验密码
9. `generateToken()` 生成 JWT
10. 返回 `user + token`

10.5 登录成功后前端做了什么

`userStore.login()` 登录成功后会：

1. 把 `user` 写入 `Pinia`；
2. 把 `token` 写入 `Pinia`；
3. 把 `token` 写入 `localStorage`；
4. 把 `user` 写入 `localStorage`。

随后 `Login.vue` 中的 `handleLogin()` 会：

1. 弹出登录成功提示；
2. 读取 `route.query.redirect`；
3. `router.push(redirect || '/dashboard')`。

10.6 登录后页面框架如何完成初始化

进入主框架后，`Layout.vue` 会加载页面容器。

同时 `NotificationCenter.vue` 会在 `onMounted()` 或弹出时调用 `loadCenter()`：

1. `notificationApi.getAll(24)` 拉取最近通知；
2. 如果当前是馆员或管理员，再调用 `borrowingApi.getPendingRenewalRequests()` 拉取待审批续借申请。

这就是为什么用户登录后右上角通知中心会立即有数据。

11. 从登录到借书：完整函数调用链

借书分两条路线，你答辩时一定要说清楚。

11.1 路线 A：管理员 / 馆员代借

这个流程发生在 `web/src/views/Borrowing.vue` 的借书页签。

前端步骤 1：搜索读者

函数：

```
searchReader()
```

执行内容：

1. 读取输入的读者编号或姓名；
2. 调用 `readerApi.search()`；
3. 后端接口是 `/api/v1/readers/search`；
4. 如果只找到一个读者，直接选中；
5. 如果找到多个，弹窗让管理员选择；
6. 最终通过 `handleSelectReader(row)` 把读者写入 `selectedReader`。

前端步骤 2：搜索图书

函数：

```
searchBook()
```

执行内容：

1. 读取输入的 ISBN 或书名；
2. 调用 `bookApi.getAll({ keyword })`；
3. 后端接口是 `/api/v1/books`；
4. 如果只找到一本书，直接选中；
5. 如果多本，弹出图书选择框；
6. 最终通过 `handleSelectBook(row)` 写入 `selectedBook`。

前端步骤 3：确认借书

函数：

```
handleBorrow()
```

它会先在前端做三层校验：

1. 是否已选读者；
2. 是否已选图书；
3. 图书库存是否大于 0。

之后调用：

```
borrowingApi.borrow(selectedReader.id, selectedBook.id)
```

11.2 后端借书链路

借书请求到达后端的完整链路如下：

1. `borrowing.routes.ts`
2. `router.use(authMiddleware)` 先登录校验
3. `router.post('/', requirePermission('borrowing:borrow'), borrowingController.borrowBook)`
4. `borrowingController.borrowBook`
5. `BorrowingService.borrowBook(readerId, bookId)`

11.3 `BorrowingService.borrowBook()` 做了什么

这是整个借书业务最核心的函数之一。

它依次做：

1. `ReaderRepository.findById(readerId)`：确认读者存在。
2. 检查读者状态是否是 `active`。
3. 检查读者证件是否过期。
4. `ReaderRepository.getBorrowingCount(readerId)`：检查当前借阅数量。
5. 比较是否达到 `max_borrow_count`。
6. `ReaderRepository.hasOverdueBooks(readerId)`：检查是否有逾期未还。
7. `BookRepository.findById(bookId)`：确认图书存在。
8. 检查图书状态是否允许借阅。
9. 检查 `available_quantity` 是否足够。
10. `BorrowingRepository.findActiveBorrowing(readerId, bookId)`：避免重复借同一本书。
11. 计算 `borrow_date` 和 `due_date`。
12. 开启数据库事务：
13. `BorrowingRepository.create(...)`：插入借阅记录。
14. `BookRepository.decreaseAvailableQuantity(bookId, 1)`：减少可借库存。

这里一定要强调“事务”。

为什么？

因为借书操作本质上是两个动作：

1. 插入借阅流水；
2. 扣减库存。

如果只成功一个，系统就会乱。

所以这里必须用事务保证一致性。

11.4 借书成功后前端做了什么

成功后，前端会：

1. 提示“借阅成功”；
2. 清空搜索框；
3. 清空已选读者和图书；
4. 调用 `searchBorrowedBooks()` 刷新当前借阅列表。

12. 普通用户借书链路

教师和学生的借书入口与馆员不同。

12.1 在图书页直接借书

文件：

`web/src/views/Books.vue`

函数：

`handleUserBorrow(book)`

它会：

1. 检查 `userStore.user?.reader_id` 是否存在；
2. 调用 `borrowingApi.borrow(userStore.user.reader_id, book.id)`；
3. 后端仍然走同一个 `BorrowingService.borrowBook()`。

12.2 在 AI 推荐面板里直接借书

文件：

`web/src/views/AIAssistant.vue`

函数：

`handleBorrow(book)`

它同样会：

1. 取当前登录用户的 `reader_id`；
2. 调用 `borrowingApi.borrow(readerId, book.id)`；
3. 借阅成功后更新推荐面板中的库存显示。

这说明一个非常重要的设计思想：

系统把“借书能力”沉淀到后端服务里，而不是绑死在某一个页面上。
所以前台图书页能借，AI 推荐卡片也能借，但最终都复用同一个借书服务。

13. 从借阅到归还、续借、审批：完整链路

13.1 归还流程

前端函数：

`Borrowing.vue` 中的 `handleReturn(row)`

调用：

`borrowingApi.return(row.id)`

后端链路：

1. `PUT /api/v1/borrowings/:id/return`
2. `borrowingController.returnBook`
3. `BorrowingService.returnBook(recordId)`

`returnBook()` 内部做：

1. 查询借阅记录；
2. 判断是否已经归还；
3. 计算逾期罚金 `calculateFine()`；
4. 开事务；
5. 更新借阅记录状态为 `returned`；
6. `BookRepository.increaseAvailableQuantity()` 增加库存。

13.2 普通读者发起续借申请

前端函数：

`Borrowing.vue` 中的 `handleRenew(row)`

调用：

`borrowingApi.requestRenewal(row.id)`

后端链路：

1. `POST /api/v1/borrowings/:id/renew-request`
2. `borrowingController.requestRenewal`
3. `BorrowingService.requestRenewal(recordId, requester, note?)`

这个服务会做：

1. 检查当前账号是否绑定 `reader_id`；
2. 禁止管理员和馆员走普通申请流程；
3. 校验借阅记录是否属于自己；
4. 校验是否允许续借；
5. 检查是否已有待处理续借申请；

6. 创建 `renewal_requests` 记录;

7. 调用 `notificationService.notifyRenewalRequest()` 给管理员/馆员发通知。

13.3 管理员审批续借

前端入口:

`NotificationCenter.vue`

核心函数:

- `approveRenewal(request)`
- `rejectRenewal(request)`

调用:

`borrowingApi.reviewRenewalRequest(request.id, 'approve' | 'reject', note?)`

后端链路:

1. `POST /api/v1/borrowings/renewal-requests/:id/review`
2. `borrowingController.reviewRenewalRequest`
3. `BorrowingService.reviewRenewalRequest()`

审批通过时内部会:

1. 再次校验申请状态;
2. 开事务;
3. 如果通过, 则调用 `performRenewal()` 延长应还日期;
4. 更新 `renewal_requests` 状态;
5. 再调用 `notificationService.notifyRenewalResult()` 给申请人发送审批结果。

这就是完整的:

申请 -> 待审批 -> 审批 -> 通知回执

闭环。

14. 从登录到 AI 对话: 完整函数调用链

这是本项目另一个重点。

14.1 AI 页面初始化

文件:

`web/src/views/AIAssistant.vue`

页面挂载时执行:

1. `checkAIStatus()`
2. `loadVectorStats()`

3. `loadConversations()`

分别用于：

- 检查 AI 服务是否在线；
- 获取向量数量和覆盖率；
- 拉取历史会话。

14.2 用户发送消息时的前端链路

主函数：

`sendMessageV2()`

调用过程：

1. 获取输入框内容；
2. 调用 `buildChatContext(chatHistory.value)` 构造最近上下文；
3. 把当前用户消息先插入 `chatHistory`；
4. 调用 `generateAssistantReply(userMessage, history)`。

14.3 `generateAssistantReply()` 做了什么

这是前端 AI 流式聊天的核心函数。

它会：

1. 先插入一个“assistant loading 消息”；
2. 把页面状态 `loading` 设为 true；
3. 调用 `aiApi.chatStream(...)`；
4. 注册多个 SSE 回调。

这些回调分别处理：

- `onChunk`：持续拼接 AI 文本；
- `onToolCall`：显示工具调用卡片；
- `onRecommend`：更新右侧推荐图书面板；
- `onComplete`：结束后保存会话；
- `onError`：提示错误。

14.4 后端流式 AI 链路

后端接口：

`POST /api/v1/ai/chat/stream`

对应文件：

`server/src/routes/ai.routes.ts`

核心调用链：

1. 校验登录 `authMiddleware`
2. 读取用户 ID 和 `reader_id`
3. 组装 `messages`
4. 注入 `SYSTEM_PROMPT`
5. 创建 OpenAI 客户端
6. 调用 `runAgentLoop(client, chatModel, messages, { userId, readerId }, res)`

14.5 `runAgentLoop()` 的本质

这个函数不是“一次请求一次回答”那么简单。
它是一个代理循环。

它会反复做下面的事情：

1. 先把消息发给模型；
2. 模型如果直接回答，就把文本流式推给前端；
3. 模型如果发起工具调用，就解析 `tool_calls`；
4. 对每一个工具调用执行 `executeTool()`；
5. 把工具结果追加回对话上下文；
6. 再次请求模型生成下一轮回答；
7. 直到没有新的工具调用，才发送 `done`。

这就是为什么 AI 页面里不仅能看到文字，还能看到“正在搜索图书”“正在借阅图书”的工具卡片。

14.6 AI 能调用哪些工具

工具定义在：

`server/src/domains/ai/tools.ts`

当前支持的核心工具有：

- `search_books`
- `recommend_books`
- `get_book_details`
- `get_borrowing_status`
- `borrow_book`
- `search_notes`
- `publish_note`
- `get_my_borrowings`
- `get_popular_books`
- `get_reader_info`

14.7 这些工具分别解决什么问题

- `search_books`：让 AI 真正查馆藏，不是瞎编。
- `recommend_books`：让 AI 结合馆藏可借情况给推荐。
- `get_book_details`：让 AI 回答某本书的详细信息。
- `get_borrowing_status`：让 AI 回答是否有库存、谁在借。
- `borrow_book`：让 AI 不只说“建议你借”，而是直接执行借阅。
- `search_notes`：让 AI 能把读书笔记内容也纳入回答。
- `publish_note`：让 AI 帮助用户沉淀笔记。
- `get_my_borrowings`：让 AI 说出“你现在借了什么”。
- `get_popular_books`：让 AI 结合近期热门排行回答。
- `get_reader_info`：让 AI 结合读者身份、上限、状态回答。

14.8 语义检索是怎么接进去的

在 `tools.ts` 的 `search_books` 语义模式里，系统会：

1. 从 `book_vectors` 里取出向量；
2. 对用户查询生成 embedding；
3. 用 `cosineSimilarity()` 计算相似度；
4. 排序后返回最相近图书。

这意味着用户不一定非要输入准确书名，
用自然语言描述需求也能找到相关图书。

14.9 AI 对话结束后发生什么

前端在 `onComplete` 阶段会调用：

```
saveCurrentConversation()
```

它会：

1. 生成会话标题；
2. 把 `chatHistory` 序列化；
3. 如果有当前会话 ID，则调用 `aiApi.updateConversation()`；
4. 否则调用 `aiApi.saveConversation()` 新建会话。

所以这个 AI 模块不仅“能聊”，还“能记住之前聊了什么”。

15. 传承笔记模块为什么是这个项目的亮点

文件核心：

- `server/src/domains/note/note.service.ts`
- `server/src/domains/note/note.repository.ts`

- `server/src/domains/note/note.routes.ts`
- `web/src/views/Notes.vue`

15.1 它不是普通笔记模块

普通笔记系统只会做：

- 新增；
- 编辑；
- 删除；
- 列表。

但这个项目的笔记分三种可见性：

1. `private`：个人笔记。
2. `public`：公开广场。
3. `legacy`：传承笔记。

15.2 传承笔记的业务规则

`NoteService.createNote()` 和 `updateNote()` 里最关键的约束是：

- 如果要创建 `legacy` 笔记，必须先绑定图书；
- 对普通读者来说，必须“当前正在借阅这本书”才允许创建；
- 系统会记录 `legacy_borrowing_id`，说明这条传承笔记与哪一次借阅行为相关。

`getNoteById()` 和 `getLegacyNote()` 里又做了可见性控制：

- 作者自己能看；
- 管理员和馆员能看；
- 普通用户只有在“当前正在借阅该书”时才能看。

15.3 这个设计解决了什么问题

它把“笔记”从私人备忘录，升级成了“与阅读行为绑定的知识传递机制”。

简单说：

- 一本书不再只是被借走又还回；
- 它还能把前一个读者的思考经验传给后一个读者。

答辩时可以这样说：

传统图书管理系统只管理图书流转，而我们的传承笔记模块把知识也纳入流转。读者借到一本书时，不只是获得书本内容，还可能接触前一位读者留下的思考，这就是“书脉”这个项目名的由来。

16. 每个模块干什么、为什么引入、答辩怎么说

模块	干什么	为什么引入	答辩时怎么说
登录模块	用户进入系统、获得身份	没有身份就无法做权限控制	我们先解决“谁能进系统”的问题
路由守卫	控制页面访问	防止未登录或越权访问页面	我们在前端做第一层访问控制
Pinia 用户状态	保存用户与 token	避免每页重复处理登录态	统一状态中心保证刷新后会话恢复
Axios 拦截器	自动带 token、自动刷新	降低页面复杂度，提高体验	token 刷新被沉淀到基础设施层
图书模块	管理馆藏与检索	图书是系统核心对象	它承担了馆藏维护和借阅入口双重职责
读者模块	管理借阅主体	登录用户不等于借阅读者	我们把账号身份和借阅身份解耦
读者类别	定义借阅规则	教师学生规则不同	借阅规则配置化而不是写死代码
借阅模块	借、还、续借、记录	图书馆核心业务闭环	它是传统馆务流程的核心引擎
事务处理	保证借书与扣库存一致	防止数据错乱	借书是多表写操作，必须事务化
RBAC 权限	不同角色做不同事	安全与职责隔离	登录和权限是两层，不是一个概念
通知中心	消息、审批、广播	续借和提醒是异步的	通知中心把业务流转做完整了
续借审批	申请-审核-回执	贴近真实图书馆管理	不是一个按钮，而是一条审批链
统计分析	展示趋势与排行	从管理工具升级成决策工具	不只记录业务，还支持分析业务
笔记模块	沉淀阅读内容	把借书行为延伸到知识沉淀	让系统从“管书”升级成“管知识”
传承笔记	让阅读经验流转	形成项目差异化创新点	让知识随着书本在读者间传递
AI 助手	自然语言入口	降低操作门槛	用户不必会用复杂菜单，也能完成操作
工具调用	AI 调用真实系统能力	防止 AI 幻觉	AI 不是编答案，而是查系统、调服务
向量检索	语义找书	支持模糊需求检索	用户描述需求也能找到相关馆藏
AI 配置模块	后台管理模型配置	部署环境不同，不能写死	AI 能力是可配置的，不依赖硬编码
导出模块	导出 CSV/JSON/报表	方便展示和管理	兼顾业务使用和答辩展示
审计日志	留痕	敏感操作可追踪	系统具备工程级可追溯性

17. 如果你要复刻这个项目，正确顺序应该是什么

不要一上来就做 AI 页面。

正确的复刻顺序应该是从“底座”到“亮点”。

第 1 步：先建数据库模型

先把这些表建出来：

- `users`
- `reader_categories`
- `readers`
- `book_categories`
- `books`
- `borrowing_records`
- `role_permissions`
- `system_settings`
- `notes`
- `renewal_requests`
- `notifications`
- `ai_conversations`
- `book_vectors`

为什么先建库？

因为后面的服务、接口、页面，全部依赖数据模型。

第 2 步：做后端基础设施

先搭：

- Express
- `app.ts`
- 错误处理中间件
- 认证中间件
- 权限中间件
- 限流中间件

理由：

先把“框架骨架”搭好，后面业务模块都能按统一规范接入。

第 3 步：先完成认证和权限

包括：

- 登录
- 注册
- Token 刷新
- `authMiddleware`
- `role_permissions`

理由：

没有身份体系，后面的页面都是假的。

第 4 步：做最核心的三大业务实体

按顺序做：

1. 图书模块
2. 读者模块
3. 借阅模块

原因：

图书馆系统最小闭环就是：

谁 -> 借了 -> 哪本书

第 5 步：做前端基础框架

包括：

- Vue3 工程
- 路由
- Pinia
- Layout
- 登录页
- 首页

第 6 步：把图书、读者、借阅页面接起来

做到这一步，系统已经是一个可用的图书管理系统了。

第 7 步：再加通知和续借审批

这是把“单次操作”升级成“业务流”的关键一步。

第 8 步：再加笔记和传承笔记

这是项目形成特色的第一层创新。

第 9 步：最后再加 AI

AI 相关应该最后接，因为它依赖：

- 图书数据已经可查；
- 借阅能力已经可调；
- 笔记数据已经存在；
- 向量数据可以生成。

第 10 步：补启动脚本、种子数据和演示数据

这样项目才能方便演示、部署、答辩。

18. 这个项目复刻时必须保留的关键设计

如果只抄页面，不保留下面这些点，就不是这个项目了。

18.1 users 与 readers 分离

这是数据建模上的关键设计。

18.2 借书事务化

借阅记录和库存更新必须放在同一事务里。

18.3 RBAC 权限

必须保留 `admin / librarian / teacher / student` 的角色分层。

18.4 自动 Token 刷新

否则用户体验会明显下降。

18.5 笔记三态模型

`private / public / legacy` 是项目特色的核心。

18.6 续借申请不是同步接口，而是审批流

这体现系统对真实业务流程的理解。

18.7 AI 不是普通聊天框，而是工具调用 Agent

AI 必须能查馆藏、查借阅、查笔记，甚至触发借书。

18.8 向量表独立

`book_vectors` 不能和 `books` 混成一张表。

18.9 AI 配置存数据库

`system_settings` 这一层很重要，它让模型配置可运营、可修改、可测试。

19. 这个项目的创新点，你可以直接这样讲

创新点 1：传承笔记

不是普通笔记，而是与借阅行为绑定的知识传递机制。

创新点 2：AI 工具调用

AI 不只是问答，而是可以访问真实馆藏数据并触发业务动作。

创新点 3：语义检索

读者不必知道精确书名，只要描述需求就能找到相关图书。

创新点 4：续借审批与通知中心

把借阅业务做成真实的流程闭环，而不是简单按钮交互。

创新点 5：账号身份与借阅身份分离

让系统建模更贴近真实图书馆管理场景。

20. 答辩高频问题与参考回答

问题 1：为什么要分前后端？

回答：

前端负责交互，后端负责业务规则和安全控制，数据库负责持久化。这样职责清晰，后期也更容易扩展 AI、移动端或管理端。

问题 2：为什么不用一个用户表同时表示读者？

回答：

因为登录身份和借阅身份不是一回事。管理员和馆员需要登录，但不一定是普通借阅读者；借阅规则还依赖读者类别，所以必须拆开。

问题 3：借书为什么要走事务？

回答：

因为借书同时涉及借阅记录写入和库存扣减，如果中间只成功一半，数据就会不一致，所以必须事务化。

问题 4：AI 为什么不直接读图书表生成回答？

回答：

因为直接让模型“记住图书数据”会过时，而且容易幻觉。我们让 AI 通过工具调用实时访问馆藏、借阅和笔记数据，这样回答更可信。

问题 5：传承笔记相比普通评论有什么不同？

回答：

评论通常是公开静态展示，而传承笔记和借阅行为绑定，后续借到这本书的人才能触达，强调的是阅读经验沿着书本流转。

问题 6：为什么 AI 配置要放到数据库？

回答：

因为部署环境、模型服务商和模型名称都可能变化。把配置放数据库后，管理员可以在设置页修改，不需要重新改代码发布。

问题 7：为什么需要通知中心？

回答：

因为续借、到期提醒、广播通知都属于异步事件，不能靠单个页面按钮解决。通知中心让业务流程和消息回执闭环。

21. 你答辩时可以直接背的两版讲稿

21.1 30 秒版本

这个项目是一个基于 Vue3、Express 和 SQLite 实现的智能图书知识链路平台。它不仅实现了传统的图书、读者、借阅和统计功能，还加入了传承笔记、AI 对话、语义检索和续借审批通知等模块。核心思想是把“图书流转”升级成“图书流转 + 知识流转 + 智能服务”。

21.2 90 秒版本

我们的系统整体采用前后端分离架构，前端使用 Vue3 负责页面交互，后端使用 Express 负责业务规则和权限控制，SQLite 负责核心数据存储。数据模型上，我们把 users 和 readers 分离，分别处理登录身份和借阅身份；业务上，以 books、readers、borrowing_records 三张核心表构成传统图书馆闭环；在此基础上加入 notes、renewal_requests、notifications 和 book_vectors 等扩展表，分别支撑传承笔记、续借审批、消息中心和语义检索。AI 部分不是简单聊天，而是通过工具调用去查馆藏、查借阅、查笔记，甚至执行借书操作，所以它是一个可执行的智能图书助手，而不是普通问答框。

22. 实现细节提醒与复刻注意点

22.1 一键启动不是纯前端开发模式

这个项目的启动脚本会：

- 处理依赖；
- 构建前端；
- 由 Express 静态托管 `web/dist`。

所以正式演示模式不是单独开一个 Vite 页面那么简单。

22.2 开发模式是前后端分离端口

常见开发模式下：

- Vite 前端在 `3000`
- Express 后端在 `3001`

并通过代理转发 `/api`。

22.3 你在讲 AI 时要强调“可配置”

因为系统的 AI 配置来自 `system_settings`，不是只依赖 `.env`。
这是一个很好的工程化表述点。

22.4 你在讲搜索时要区分三种

项目不是只有一种搜索：

- 普通关键字搜索
- 正则搜索
- 语义向量搜索

这个层次说清楚，评委会觉得你理解得比较深入。

23. 最后给你三句最值得记住的话

第一句

这个系统的底座是“图书、读者、借阅”三大核心业务。

第二句

这个系统的特色是“传承笔记 + AI 工具调用”，让图书流转升级为知识流转和智能服务。

第三句

这个系统的架构重点是前后端分层、权限分层、业务事务一致性，以及 AI 与真实业务能力的解耦集成。

24. 复刻本项目的最小检查清单

如果你要确认自己复刻得像不像，检查下面这些点：

- 能否登录并按角色进入不同页面
- 刷新页面后登录态是否保留
- 图书、读者、借阅三大基础模块是否打通
- 借书是否真的会减少可借库存
- 还书是否真的会恢复可借库存
- 普通读者是否只能看到自己的借阅数据
- 续借是否经过申请、审批、通知三步
- 笔记是否支持 private / public / legacy 三态
- 传承笔记是否要求当前借阅该书才能创建或查看
- AI 是否能调用真实数据，而不是纯文本聊天
- 语义检索是否依赖 `book_vectors`
- AI 配置是否可以从后台修改

如果这些都具备了，你复刻出来的就已经不是一个“页面相似”的项目，而是一个“架构和业务都复刻到位”的项目。